Lesson Activities — 2.2 Problem Solving & Programming

In-lesson classroom activities for OCR A Level Computer Science H446, Component 02, Section 2.2 (2.2.1 Programming techniques · 2.2.2 Computational methods) — unplugged demos, Parson's problems, role plays and prediction tasks, not exam questions.

2.2.1 Programming techniques

Russian Dolls & the Tray Stack (unplugged demo · 15 min · whole class)

Resources: A set of Russian dolls (or 4 nested boxes/envelopes); 4 canteen trays (or 4 A4 "stack frame" sheets); marker pens; 4 volunteer students; sticky notes.

How it runs: 1. Open the Russian dolls one by one: "each doll contains a smaller version of the same problem — that is recursion. The tiny solid doll in the middle is the **base case**: it contains nothing, so opening stops." Reassemble them to show the returns happening in reverse. 2. Now make it precise with `factorial(4)`. Write on the board:

```
function factorial(n)
    if n == 1 then
        return 1
    else
        return n * factorial(n - 1)
    endif
endfunction
print(factorial(4))
```

3. Give each of 4 volunteers a tray. Student 1 writes `n = 4` and "return address: print" on a sticky note, puts it on their tray and holds it. They cannot answer yet — they need `factorial(3)` — so Student 2 stacks their tray (`n = 3`, "return to: Student 1's line") **on top**. Repeat for `n = 2` and `n = 1`. Physically stack all 4 trays in a pile (or students stand in a line, each resting hands on the previous person's shoulders).
4. Student 4 hits the base case: `n == 1`, so they announce "return 1", take their tray **off the top** and sit down.
5. Unwind: Student 3 computes `2 × 1 = 2` and pops off; Student 2 computes `3 × 2 = 6`; Student 1 computes `4 × 6 = 24` and shouts the final answer.
6. Ask: "What order did trays go on? What order did they come off?" (LIFO — this pile IS the call stack; each tray is a stack frame holding a parameter, locals and a return address.)
7. Finally, cross out the base case on the board and ask what happens to the tray pile. (It grows forever — until the canteen runs out of trays: **stack overflow**.)

Answers / expected outcomes: - Frames pushed: `factorial(4)`, `factorial(3)`, `factorial(2)`, `factorial(1)` — popped in reverse (LIFO). - Returns in order: `1`, then `2×1 = 2`, then `3×2 = 6`, then `4×6 = 24`. Final output: **24**. - No base case → frames pushed forever → **stack overflow**. - Students should use the vocabulary: *base case, recursive/general case, stack frame, push, pop, return address, LIFO, stack overflow*.

Stretch: Trace `factorial(0)` with this code (base case `n == 1` is never reached for n = 0 — it overflows; fix by changing the base case to `n <= 1` or `n == 0`). Then challenge students to rewrite factorial iteratively and state the memory trade-off (one set of variables vs one frame per call).

Parson's Problem: The Bouncer's Loop (Parson's problem · 10 min · pairs)

Resources: One envelope per pair containing the 8 code strips below, pre-shuffled (print, cut into strips). One strip is a deliberate distractor.

How it runs: 1. Brief: "This program must keep asking for an age until it gets one between 11 and 18 inclusive, telling the user off after each bad attempt. It must ask **at least once**. Rebuild it — one strip is a red herring and must be left out." 2. Hand out the shuffled strips: - **A** `until age >= 11 AND age <= 18` - **B** `age = int(input("Enter age (11-18): "))` - **C** `do` - **D** `print("Thanks, age recorded: " + str(age))` - **E** `if age < 11 OR age > 18 then` - **F** `print("Out of range - try again")` - **G** `endif` - **H** `while age >= 11 AND age <= 18` *(distractor)* 3. Pairs arrange strips; when they think they're done, they must trace inputs `9, 25, 14` on a mini-whiteboard before you check their order. 4. Debrief: why is `do…until` the right loop here, not `while`? (Post-conditioned — the body always runs at least once, so the user is always asked; a pre-conditioned `while` would test `age` before it has a sensible value.)

Answers / expected outcomes: - Correct order: **C, B, E, F, G, A, D** (H is the distractor — it is a pre-condition test and its logic is inverted relative to the `until`). - Trace for inputs 9, 25, 14: `Out of range - try again` → `Out of range - try again` → `Thanks, age recorded: 14`. - Key vocabulary surfaced: condition-controlled iteration, post-conditioned loop, validation.

Stretch: Rewrite the same validation using `while` (needs a priming input before the loop, or a flag such as `valid = false`). Which version has less repeated code? Then extend: reject non-numeric input too.

Sticky Notes vs the Shared Whiteboard (unplugged demo · 10 min · whole class)

Resources: Sticky notes; a whiteboard (or one large flipchart sheet) labelled `playerScore`; marker; this code on the board:

```
procedure addBonus(score:byVal)
    score = score + 10
    print("inside: " + str(score))
endprocedure

playerScore = 50
addBonus(playerScore)
print("after: " + str(playerScore))
```

How it runs: 1. Write `50` on the whiteboard under the label `playerScore` — this is the variable in memory. 2. **Round 1 — by value (`byVal`).** Call a volunteer "the subroutine". Copy the value onto a **sticky note** (`score = 50`) and hand it to them: "you got a *copy*". They add 10 **on the sticky note only** (60) and announce "inside: 60". When the subroutine ends, theatrically screw up the sticky note and bin it (local lifetime over). Ask the class to predict the second print, then reveal the whiteboard: still 50. 3. **Round 2 — by reference (`byRef`).** Change the signature to `score:byRef`. This time hand the volunteer a sticky note with **an arrow drawn on it pointing at the whiteboard** — "you got the *address*, not a

copy". They walk to the whiteboard and change 50 to 60 directly. Bin the arrow note at the end — but the whiteboard now says 60. 4. Class writes one sentence for each round starting "The original was…".

Answers / expected outcomes: - By value: `inside: 60` then `after: 50` — a **copy** is passed, the **original is unchanged**. - By reference: `inside: 60` then `after: 60` — the **memory address** is passed, so the **original can be changed**. - The exam-worthy sentence is the *consequence* ("the original is unchanged / can be changed"), not just "a copy is made".

Stretch: When is by reference the better choice even though it's less safe? (Large data structures — no expensive copy; or when the subroutine must send back more than one change.) What happens with `addBonus(50)` — passing a literal by reference? (Nothing sensible to point at — arguments passed by reference must be variables.)

Rooms in a House (role play · 15 min · groups of 4-6)

Resources: Classroom zoned with masking tape: a central "hallway" (main program) with a large **noticeboard** sheet, plus two taped "rooms" labelled `procedure kitchen()` and `procedure garage()`, each containing a small personal whiteboard; sticky notes; a bin in each room.

How it runs: 1. Set the metaphor: the **house = the program**, the **hallway noticeboard = a global variable** (visible from every room through the open doors), a **room = a subroutine**, and anything written on a room's whiteboard = a **local variable** that is binned when you leave the room. 2. Pin `total = 100` on the hallway noticeboard. Send a student into `kitchen()` to run this script (on the board):

```
total = 100

procedure kitchen()
    total = 5
    print(total)
endprocedure

kitchen()
print(total)
```

Inside the room they write `total = 5` on the **room whiteboard** (a new local that **shadows** the global). For `print(total)` they must read the *nearest* `total` — the room's own whiteboard. 3. They leave the room; the room whiteboard is wiped (local lifetime ends). Back in the hallway, the class runs the final `print(total)` reading the noticeboard. 4. Now send a student into `garage()` with `snack = "toast"` written on the room whiteboard. When they leave, the note is binned. Back in the hallway, try to run `print(snack)` — the class must rule on what happens. 5. Groups then answer three "estate agent" questions on mini-whiteboards: Where must a variable live to be seen by every room? Why is it risky to keep everything on the noticeboard? Which variables survive the whole run?

Answers / expected outcomes: - Step 2-3 output: `5` then `100` — the assignment inside the subroutine creates a **local** that **shadows** the global; the global is untouched. - Step 4: `print(snack)` in the hallway is an **error** — `snack` is out of scope (local to `garage()`, destroyed on exit). - Discussion points: globals are visible everywhere and live for the whole run, but overusing them causes side effects and makes debugging/maintenance harder; prefer locals for modularity and memory freed on exit.

Stretch: Add a second call to `kitchen()` — does the local `total` remember its old value? (No — fresh frame each call.) How could a room legitimately change the noticeboard? (Declare/use it as `global`, or better: pass it in and return the new value — discuss why the latter is preferred.)

What Prints Next? (live-coding prediction · 12 min · individual, mini-whiteboards)

Resources: Mini-whiteboards and pens; IDE or OCR-ERL simulator on the projector (or reveal line-by-line on slides).

How it runs: 1. Show Round 1 with the output hidden:

```
x = 10
while x > 20
    print(x)
    x = x - 2
endwhile
print("A")
do
    print(x)
    x = x - 2
until x < 6
print("B")
```

2. Before running anything, students write on whiteboards: *how many lines will the `while` loop print?* Hold boards up; run it; discuss (zero — pre-conditioned, test fails immediately).
3. Then step through the `do…until` one iteration at a time. Before each iteration, boards up: "what prints next?" Reveal after each prediction.
4. Round 2 — OOP polymorphism, same routine:

```
class Animal
    public procedure speak()
        print("...")
    endprocedure
endclass

class Dog inherits Animal
    public procedure speak()
        print("Woof")
    endprocedure
endclass

pets = [new Animal(), new Dog(), new Animal()]
for i = 0 to 2
    pets[i].speak()
next i
```

Boards up before each loop iteration: "what prints next, and why?"

Answers / expected outcomes: - Round 1 full output, in order: `A`, `10`, `8`, `6`, `B`. The `while` body runs **zero** times (pre-conditioned; 10 > 20 is false). The `do…until` prints 10, 8, 6 — it stops after x becomes 4 because `4 < 6` is true; a post-conditioned loop always runs **at least once**. - Round 2 output: `...`, `Woof`, `...` — the same call `pets[i].speak()` behaves differently per object type: **polymorphism** via Dog **overriding** the inherited method. - Watch for the classic misconception that the `while` prints `10` once.

Stretch: Change Round 1's first line to `x = 30` and re-predict both loops (while prints 30, 28, 26, 24, 22 then stops when x = 20; then A; do…until prints 20, 18, …, 6 stopping when x = 4; then B). For Round 2, add `class Cat inherits Animal` with **no** `speak()` — what does `new Cat().speak()` print? (`...` — inherited unchanged.)

2.2.2 Computational methods

The Greetings-Card Pipeline (unplugged demo · 15 min · whole class)

Resources: 12 sheets of A5 paper; 3 rubber stamps or coloured pens; envelopes; a desk arranged as a 3-station production line: **Fold** → **Stamp** → **Envelope**; stopwatch (or count "ticks" out loud).

How it runs: 1. **Run A — no pipeline.** One student makes 6 cards alone, doing all three stages per card. Class counts stages out loud: each card takes 3 "ticks", one stage per tick. 2. **Run B — pipelined.** Three students sit in a line, one stage each. Every tick, each student does their stage and passes the card on. Card 2 starts folding while card 1 is being stamped — different stages work on **different items simultaneously**. 3. Class tallies total ticks for each run on the board and computes the speed-up. 4. Ask the two key questions: "Did any *single* card get made faster?" and "So what exactly improved?" (Throughput, not latency.) 5. Map to the CPU: Fold/Stamp/Envelope ↔ **fetch/decode/execute**. What real-world event ruins a pipeline? (A stage that stalls — e.g. the stamper runs out of ink; in a CPU, a branch means the fetched instructions may be thrown away — a *flush*.)

Answers / expected outcomes: - Run A (sequential): 6 cards × 3 stages = **18 ticks**. - Run B (pipelined): first card finishes at tick 3, then one card per tick: 6 + 3 − 1 = **8 ticks**. Speed-up = 18 ÷ 8 = **2.25×**. - Each individual card still takes 3 ticks — pipelining raises **throughput**, not per-item speed. - General formula worth boarding: n items, k stages, 1 tick per stage → n + k − 1 ticks.

Stretch: What is the theoretical maximum speed-up of a k-stage pipeline as n grows? (Approaches k×.) What if the Stamp stage takes 2 ticks while the others take 1? (The slowest stage sets the pace — the pipeline runs at the speed of its bottleneck.)

Which Method Would You Hire? (card sort · 12 min · pairs)

Resources: Per pair: 10 blue **method** cards and 10 yellow **scenario** cards, shuffled (print and cut).

Method cards: *Problem recognition · Decomposition · Divide and conquer · Abstraction · Backtracking · Data mining · Heuristics · Performance modelling · Pipelining · Visualisation*

Scenario cards: 1. Solving a Sudoku by trying a digit, and undoing it and trying another when a dead end is reached 2. A supermarket analyses millions of loyalty-card baskets and discovers two products are often bought together 3. A satnav quickly picks a "good enough" route using straight-line distance estimates rather than checking every possible route 4. Simulating Black Friday traffic on a web server before launch, instead of finding out the hard way 5. A CPU fetches the next instruction while decoding

one and executing another 6. Plotting reported infection cases on a heat map, which reveals a cluster around one water pump 7. Splitting a game project into separate input, physics and rendering modules for three developers 8. Finding a name in a sorted phone book by repeatedly halving the section that could contain it 9. The London Tube map shows the order of stations but deliberately ignores real distances and geography 10. Before writing any code, the team lists the inputs, outputs, constraints and success criteria of the task

How it runs: 1. Pairs match each scenario to exactly one method (one-to-one). 2. Two deliberate near-misses to argue about: 7 vs 8 (decomposition = *different* sub-tasks; divide and conquer = *smaller instances of the same problem*, solved and combined) and 1 vs 3 (backtracking is systematic and exact; a heuristic trades optimality for speed). 3. Check by "justify one match" cold-calls: pairs must say **why** the method suits the scenario, not just name it — that "why" is where exam marks live.

Answers / expected outcomes: 1 → Backtracking · 2 → Data mining · 3 → Heuristics · 4 → Performance modelling · 5 → Pipelining · 6 → Visualisation · 7 → Decomposition · 8 → Divide and conquer · 9 → Abstraction · 10 → Problem recognition.

Stretch: For each method, pairs write one *new* scenario of their own that is not a paraphrase of the given one; hardest pairs get "heuristics" and must include the phrase "good enough, not guaranteed optimal".

Dead End! The Human Backtracking Maze (unplugged demo · 15 min · whole class)

Resources: Masking tape 4×4 grid on the floor (or 16 desks/chairs marking cells); "wall" cells covered with paper crosses; a ball of string or a stack of paper "breadcrumbs"; one navigator student, one "algorithm caller" student.

Maze layout (rows 1-4 top to bottom, columns A-D left to right; `#` = blocked):

```
row 1:   S  .  .  #
row 2:   #  #  .  #
row 3:   #  .  .  .
row 4:   #  .  #  G
```

Start S = (1,A); Goal G = (4,D).

How it runs: 1. Give the navigator the fixed rule (this makes it an *algorithm*, not wandering): **at each cell, try moves in the order West, South, East, North; never re-enter a cell already on your string.** They unwind string behind them as they walk. 2. The caller announces each decision aloud ("At (3,C): West is open — go West"). 3. When the navigator hits a dead end (nowhere new to go), they must **backtrack**: rewind the string to the most recent cell that still has an untried direction, and try the next option. The class shouts "BACKTRACK!" each time. 4. On reaching G, lay the final string route and compare it with everywhere visited. 5. Debrief: the string is exactly the **call stack** — walking forward = pushing frames, backtracking = popping. Connect to the KO definition: build a solution step by step; on a dead end, return to the last decision point and try another option (N-Queens, Sudoku — often implemented recursively).

Answers / expected outcomes: - Exploration with the W-S-E-N rule: S(1,A) → (1,B) → (1,C) → (2,C) → (3,C) → **(3,B)** → **(4,B) = dead end** → backtrack to (3,B), no options → backtrack to (3,C) → (3,D) → (4,D) = **G**. - Exactly one backtrack episode occurs (out of (4,B) back to (3,C)). - Final solution path (the string): (1,A) → (1,B) → (1,C) → (2,C) → (3,C) → (3,D) → (4,D) — 6 moves. - Key learning: backtracking is exhaustive and systematic — it *will* find a path if one exists, unlike a heuristic guess.

Stretch: Re-run with rule order E-S-W-N — does the dead end still happen? (No: from (3,C) East is tried first, going straight to (3,D) → (4,D); zero backtracks — the rule order changes the work done, not the correctness.) Then ask how the navigator would prove "no path exists" (string fully rewound to S with all options exhausted).

Always, Sometimes, Never: Computational Methods (always-sometimes-never · 10 min · whole class then pairs)

Resources: Three corner posters: ALWAYS / SOMETIMES / NEVER; statement slides; mini-whiteboards for justifications.

How it runs: 1. Reveal one statement at a time; students move to (or vote for) a corner, then one student per corner defends their choice before you adjudicate. 2. Statements: - **S1** "A heuristic gives the optimal answer." - **S2** "Abstraction removes every detail of the problem." - **S3** "Divide and conquer splits a problem into smaller instances of the *same* problem." - **S4** "Decomposition produces sub-problems that are smaller versions of the original problem." - **S5** "Pipelining makes an individual item finish faster." - **S6** "Performance modelling requires building the real system first." - **S7** "Backtracking is implemented using recursion." 3. Pairs then write one *corrected* version of each NEVER statement so it becomes ALWAYS.

Answers / expected outcomes: - **S1 Sometimes** — a heuristic *may* land on the optimum, but it is not guaranteed; it trades accuracy for speed. - **S2 Never** — abstraction removes *unnecessary* detail only; remove everything and there is no model left. - **S3 Always** — that is its definition (binary search, merge sort), which is exactly what separates it from general decomposition. - **S4 Sometimes** — only when the decomposition happens to be divide and conquer; usually the sub-problems are *different* sub-tasks. - **S5 Never** — per-item time (latency) is unchanged; throughput improves because stages overlap on different items. - **S6 Never** — the whole point is predicting behaviour with a model *before/instead of* building it. - **S7 Sometimes** — naturally recursive, but any recursion can be rewritten iteratively with an explicit stack.

Stretch: Students author one new ASN statement each about 2.2.2 content and trial it on their partner; the best three get used as next lesson's starter. (Quality bar: it must be genuinely arguable — not trivially always/never.)

The Big Map of Methods (concept mapping · 15 min · groups of 3)

Resources: A3 paper per group; sticky notes pre-printed with the node terms; glue sticks; coloured pens for labelled arrows.

Node terms: *problem recognition, decomposition, divide and conquer, abstraction, recursion, backtracking, heuristics, data mining, performance modelling, pipelining, visualisation, binary search, merge sort, A*, call stack, intractable problem*.

How it runs: 1. Groups arrange all 16 sticky notes on A3 and draw **labelled** arrows — the rule is that every arrow must carry a verb phrase ("is used by", "pairs with", "is an example of"). Unlabelled lines score nothing. 2. Minimum bar: 12 labelled links; every node connected. 3. After 10 minutes, groups rotate one desk clockwise and must add two links the other group missed (different colour pen). 4. Whole-class share of the most interesting link found in someone else's map.

Answers / expected outcomes: Any accurate labelled links accepted. Strong links to look for: - *divide and conquer* —pairs with→ *recursion*; —is used by→ *binary search* and *merge sort*. - *recursion* —relies on→ *call stack*; *backtracking* —is often implemented with→ *recursion*. - *heuristics* —provide h(n) for→ *A**;

—give good-enough answers to→ *intractable problem*. - *data mining* —often presents results via→ *visualisation*; *performance modelling* —avoids→ building the real system (annotation). - *problem recognition* —comes before→ *decomposition*; *abstraction* —removes unnecessary detail from→ the model (annotation). - Misconception to catch: an arrow equating *decomposition* and *divide and conquer* — must be labelled with the distinction (same-problem instances vs different sub-tasks).

Stretch: Add three nodes from earlier topics (*stack overflow, FDE cycle, Travelling Salesman*) and integrate them correctly (e.g. *pipelining* —speeds up→ *FDE cycle*; *TSP* —is→ *intractable* —so we use→ *heuristics*).